

CSC3060 Project 3 Report

Student Name Chaoyi, Sun

Student ID 124090550

1 Static Latency Model

(a) Load instructions must access data memory and then write back to the register file, so they need to wait for the memory fetch to complete before execution can continue. While store instructions only writes the data into a store buffer and does not need to wait for the memory write to finish.

(b) Assigning a fixed latency to each instruction type approximates the average execution behavior, which simplifies performance analysis and helps the scheduler plan for long-latency instructions like load instructions.

2 Dependency Detection

(a) Multiple instructions can read the same register simultaneously without affecting each other, since reading does not modify the register's value. Therefore, no Read-After-Read (RAR) dependency exists.

(b) If a cycle exists, the instructions would depend on each other circularly and none of them could execute first, which implies that no valid topological order can be formed.

(d) **[Bonus Part]** The single-pass DAG builder is implemented in the following function:

```
std::vector<DAGNode> DAGBuilder::build(const std::vector<Instruction>& instructions)
```

3 Priority Calculation

(a) WAR and WAW are ordering constraints rather than data dependencies. The successor only needs to wait for the predecessor to issue, without waiting for it to finish.

4 Tie-Breaking Policies

(a) This greedy strategy prioritizes instructions with more children so that more dependent instructions can become ready earlier.

(b) When instructions have significantly different execution latencies, longer-latency instructions should be scheduled earlier because they are more likely to become performance bottlenecks.

5 Complexity Analysis

(a) $O(n^2 + e)$.

Each iteration scans all n nodes to find ready nodes, and there can be up to n cycles in the worst case, resulting in $O(n^2)$ for scanning. Processing all dependency edges contributes $O(e)$ time in total. Therefore, the overall time complexity is $O(n^2 + e)$.

(b) $O(n + e)$.

With recursive memoization, the priority of each node is computed only once. Each node and each dependency edge is visited at most once during the recursion, resulting in a total time complexity of $O(n + e)$.

(c) **[Bonus Part]** $O((n + e) \log n)$.

The optimized scheduler maintains a max-heap for ready nodes and a min-heap for inflight instructions ordered by their finish cycle. Each node is inserted into the ready heap when it becomes ready and removed when issued. Since each node and dependency edge is processed at most once, there are $O(n + e)$ heap operations in total, each costing $O(\log n)$. Therefore, the overall time complexity is $O((n + e) \log n)$.

6 Run Schedule

Table 1: Performance Comparison of Different Scheduling Policies

Example	Policy	Original Cycles	Scheduled Cycles	Improvement
Example 1	SMALLER_INDEX	16	16	0%
	LPT	16	16	0%
	MOST_CHILD	16	16	0%
Example 2	SMALLER_INDEX	7	7	0%
	LPT	7	7	0%
	MOST_CHILD	7	7	0%
Example 3	SMALLER_INDEX	17	14	17.6%
	LPT	17	14	17.6%
	MOST_CHILD	17	14	17.6%
Example 4	SMALLER_INDEX	21	16	23.8%
	LPT	21	16	23.8%
	MOST_CHILD	21	16	23.8%

Scheduling improves performance only when parallelism exists. Example 1 is already optimal, and Example 2 has fully serial dependencies, so no improvement is possible. Example 3 saves 3 cycles (17.6%) by grouping loads before their uses. Example 4 saves 5 cycles (23.8%) by issuing the long-latency `div` early to overlap with independent instructions.

These results illustrate a fundamental principle: scheduling increases throughput by hiding latency, but does not reduce the latency of individual instructions (each `lw` still takes 5 cycles, and each `div` still takes 10 cycles). The performance gain comes from overlapping independent work, not from speeding up single instructions.

7 Appendix

It is hereby declared that Generative AI tools were utilized in this project to assist with code optimization, report polishing, and test case generation. The core logic and implementation were verified by the author.

[Link](#)